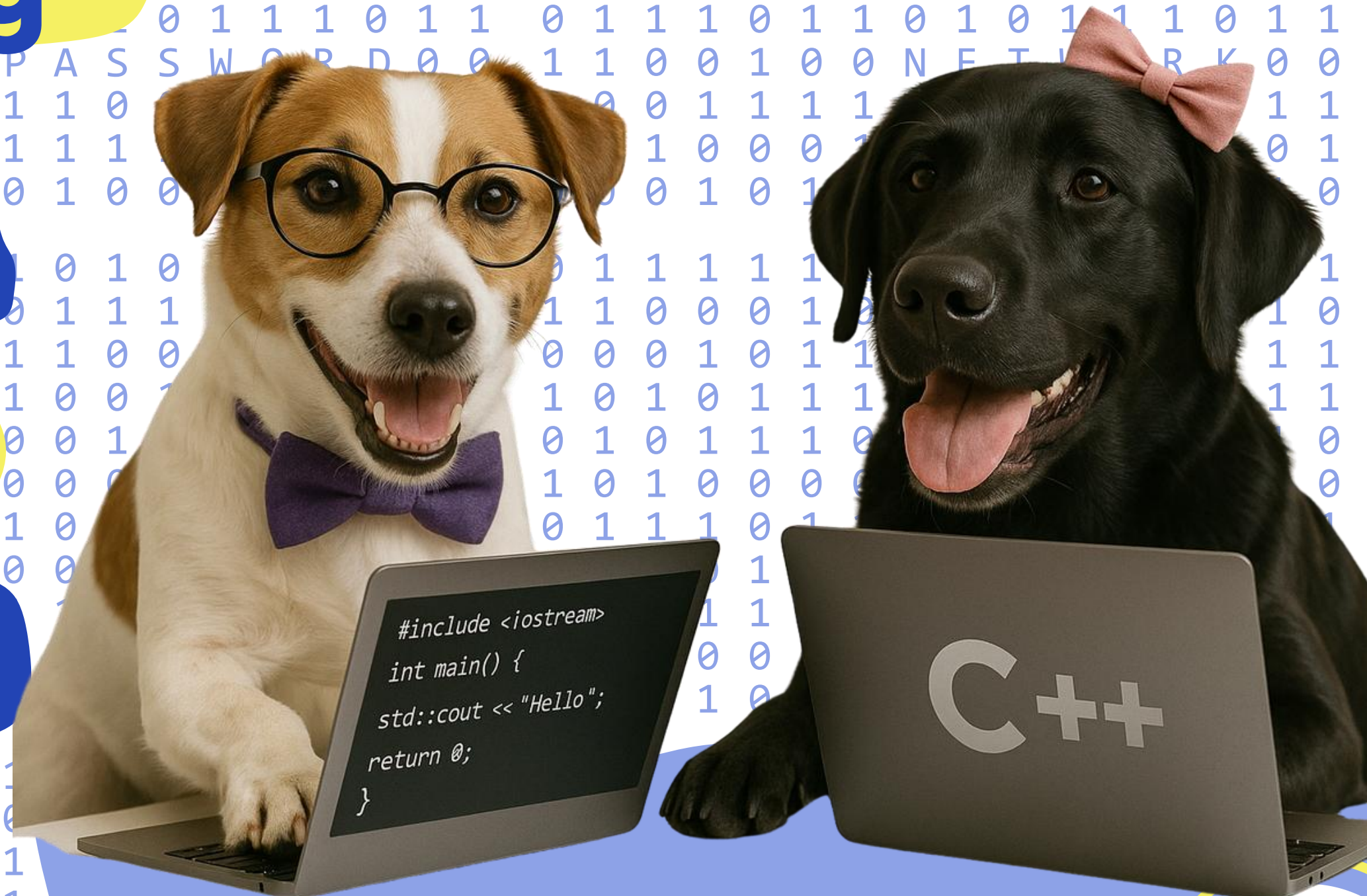


Parallel Processing

in C++

With OpenMP

How to Make Your Programs Faster!



What is OpenMP?

OpenMP (Open Multi-Processing) is an **API** (Application Protocol Interface) that supports multi-shared-memory multiprocessing programming in **C/C++**.

It allows programmers to **parallelize code** by adding **compiler directives** (pragmas) to their existing code. **#pragma**

⚡ **Goal:** Speed up programs by dividing work across multiple CPU cores



Why Parallel Processing?

Traditional programs run **sequentially**
(one instruction at a time)

Modern CPUs have **multiple cores** – let's
use them!

Example: Sorting a million words

- **Sequential:** 6.1 seconds
- **Parallel:** 1.2 seconds 🚀



Using OpenMP



```
#include <omp.h>
```

In your source or header file, you include the OpenMP library along with your other #includes and it will **provide OpenMP functionality.**

Threads

The concept of a **thread** is foundational to understanding the basics of parallel programming.

- A **thread** is like a mini-program running inside your program.
- Each thread runs part of the code concurrently (or truly in parallel, if you have multiple CPU cores).
- Threads **share the same memory** (unlike processes).



Key Features of OpenMP

Directives: OpenMP uses `#pragma omp` directives to specify parts of the code that should run in parallel (**parallel regions**), loop parallelization, and other parallel constructs.

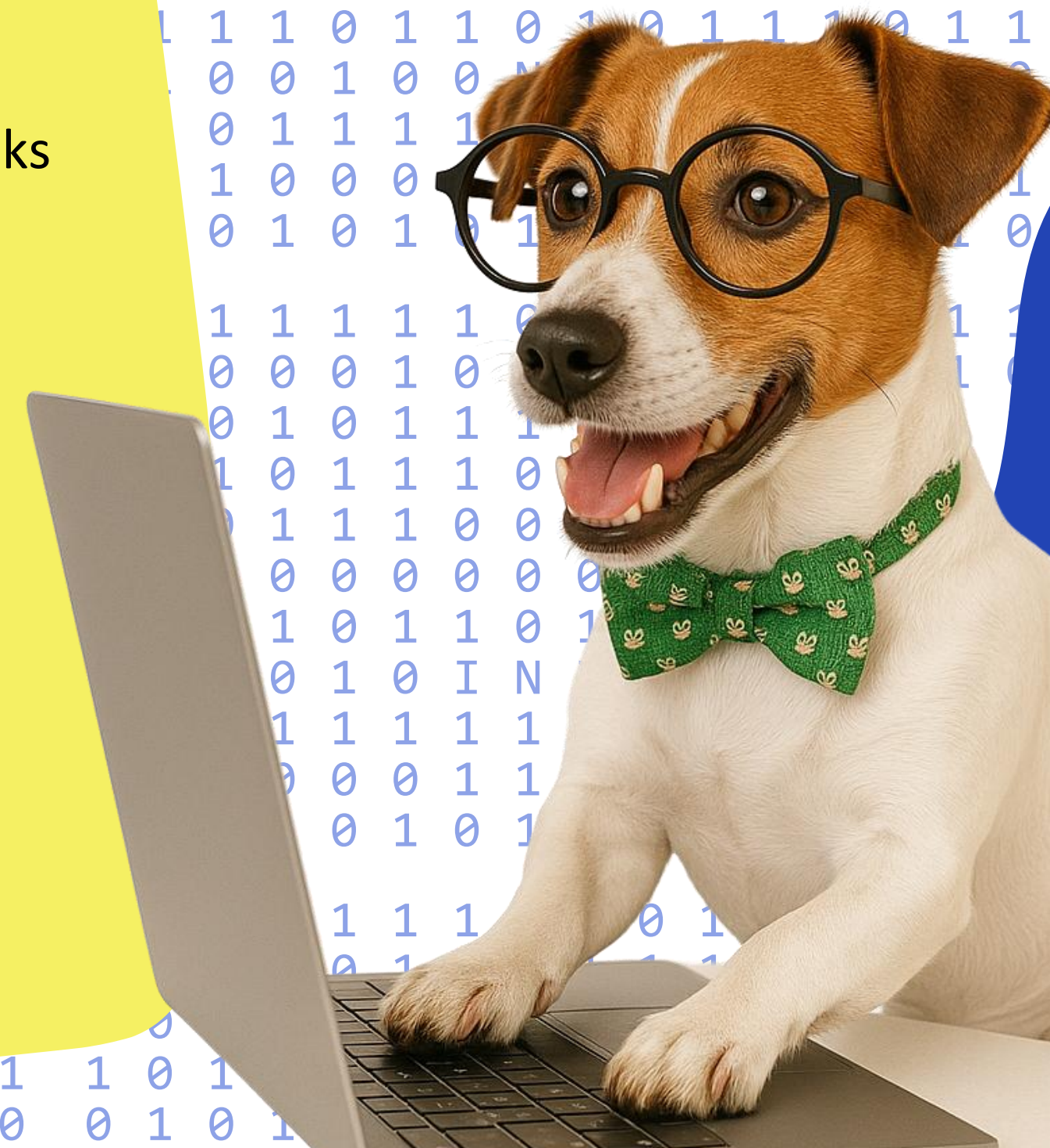
Parallel Regions: Code blocks marked with `#pragma omp parallel` defines a **parallel region** and the blocks are executed by **multiple threads**.

Work-Sharing Constructs: Directives like `#pragma omp for` distribute loop iterations among **threads**.

Data Sharing: Variables can be shared among **threads** or private to each thread.

Synchronization: OpenMP provides mechanisms for synchronizing **threads**, such as barriers and critical sections.

Thread Management: OpenMP manages the creation and management of **threads**.



Windows vs. Mac



Windows

If you are using the Windows operating system and the GCC compiler (like MinGW), then it already supports OpenMP by default. You only need to include `-fopenmp` when compiling and include the correct libraries.

Compile: `g++ -fopenmp sourcefile.cpp -o run`

Mac

Using OpenMP on a Mac requires a bit of setup because Apple's default Clang compiler does not support OpenMP by default. You'll need to install a version of the compiler that does support OpenMP, like **GCC via Homebrew**.

Directions on installing Homebrew and then GCC on next slide.

Using OpenMP on a Mac



Is Homebrew already Installed?

Open Terminal and run

```
which -s brew
```

If Homebrew is installed, you will see the full path to the brew executable, usually:

```
/opt/homebrew/bin/brew (if on Apple Silicon Macs) or
```

```
/usr/local/bin/brew (if on Intel Macs)
```

If Homebrew is not installed, you will see:

```
brew not found
```

Install Homebrew (if it isn't already installed)

In the Terminal, run

```
/bin/bash -c "$(curl -fsSL
```

```
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Install GCC (which supports OpenMP)

```
brew install gcc
```

This installs a GCC version like gcc-13 or later, depending on the current release.



Using OpenMP on a Mac, continued



Install the OpenMP Runtime Library (libomp)

```
brew install libomp
```

This installs a GCC version like gcc-13 or later, depending on the current release.

Check the GCC version

```
ls /opt/homebrew/bin | grep "g++" (if on Apple Silicon Macs) or  
ls /usr/local/bin | grep "g++" (if on Intel Macs)
```

Compiling C++ with OpenMP

If your source file is main.cpp and if Homebrew installed GCC 14, the command will be:

```
g++-14 -fopenmp main.cpp -o run
```

Replace g++-14 with the version Homebrew installed (gcc-12, gcc-14, etc.)

Run your program

```
./run
```



OpenMP Basic Syntax

```
#include <omp.h>
```

The library required to use OpenMP

```
#include <omp.h>
```

```
int main() {
```

```
#pragma omp parallel
```

```
{
```

*// Code to be executed in parallel by
multiple threads*

```
int thread_id = omp_get_thread_num();
```

```
// ...
```

```
}
```

```
return 0;
```

```
}
```

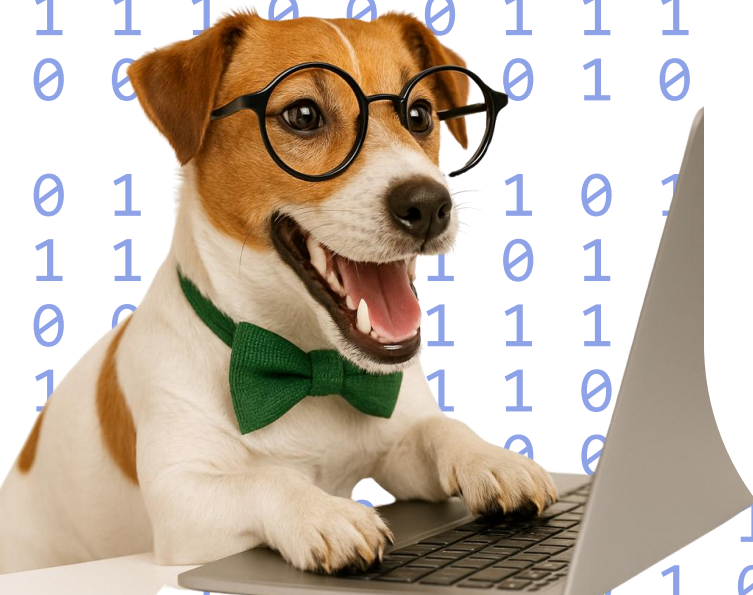
```
#pragma omp parallel
```

defines a **parallel region** and creates a team of threads to execute the enclosed code.

Note that the beginning curly brace MUST be on the next line. It can't be on the same line as any preprocessor directive (anything starting with a '#')

```
omp_get_thread_num()
```

Is an OpenMP function that returns the unique ID number of the thread currently executing the code



Setting Number of Threads

```
#include <thread>
```

The library used for calculating number of logical processors on your computer

```
#include <omp.h>
```

```
#include <thread>
```

```
int main() {  
    const auto processor_count = thread::hardware_concurrency();  
    int num_threads = 10;
```

```
    omp_set_dynamic(false);
```

```
    omp_set_num_threads(num_threads);
```

```
    #pragma omp parallel
```

```
{
```

```
        //parallel region
```

`omp_set_num_threads(int)`
Tells OPeNMP to use `int` threads in the next parallel region.

`Thread::hardware_concurrency()`
determines the number of processors on your computer. You could then print this value out to the screen.

`omp_set_dynamic(false)`
By default, OpenMP may adjust the number of threads at runtime to optimize performance based on system load and resources. But if you want full control over how many threads your program uses, you set dynamic to false (or 0).

parallel_1.cpp

Example

```
1 #include <iostream>
2 #include <vector>
3 #include <omp.h>
4 using namespace std;
5 int main() {
6     // Step 1: Create a vector with 100 elements
7     vector<int> data(10000, 1); // Fill with 1s
8
9     // Step 2: Set number of threads and disable dynamic adjustment
10    omp_set_dynamic(false); // Don't let OpenMP override thread count
11    omp_set_num_threads(10); // We want exactly 10 threads
12
13    // Step 3: Define a parallel region and operate on the vector
14    #pragma omp parallel
15    {
16        cout << "Hello from thread " << omp_get_thread_num() << "\n";
17        #pragma omp for
18        for (int i = 0; i < data.size(); i++) {
19            data[i] += static_cast<int>(omp_get_thread_num()); // Add thread ID to each element
20        }
21    }
22
23    // Step 4: Print elements to see results
24    cout << "After processing:\n";
25    for (int i = 0; i < 10000; i++) {
26        cout << data[i] << " ";
27    }
28    cout << endl;
29    return 0;
30 }
```

Look at next slide



Parallel For Loop

In OpenMP in just a **single** line, you can identify a parallel region to parallelize a for loop:

```
#pragma omp parallel for
for (int i = 0; i < 1000; i++) {
    // work here can happen in multiple threads
}
```

This tells the compiler

“Create a team of threads and split the loop work among them.”

The threads share the same memory.



Private or Shared data

In OpenMP, you can make data either **private** or **shared**.

SHARED

```
#pragma omp parallel for shared(vec)
for (int i = 0; i < vec.size(); i++) {
    vec[i] = ...; //modify element
}
```

The shared(vec) clause tells the compiler and runtime that the variable vec should be shared among all threads in the parallel region so that all threads see and operate on the same instance of vec.

PRIVATE

```
#pragma omp parallel for private(vec)
for (int i = 0; i < vec.size(); i++) {
    vec[i] = ...; //modify element
}
```

If you accidentally made a variable like vec private, each thread would get its own uninitialized copy, and the loop would either **crash** (due to out-of-bounds access) or **do nothing useful** (because it isn't working on real data). Any updates to the vector would be lost after the parallel region ends.

Date and Time Library to Measure Speedup

<chrono> library

If you want to measure the difference in time to run a sequential vs. parallel algorithm, the best library to use is <chrono>, which is the date and time library.

This library is useful if you want an accurate measurement of elapsed time.

To time an algorithm, you first include the library:

```
#include <chrono>
```



<chrono> library

```
1 #include <iostream>
2 #include <chrono>
3 using namespace std;
```

Include the <chrono> library!

```
4
5 int main() {
6
7     auto start = chrono::high_resolution_clock::now();
8
9     //all the code you are timing
10    //this sample program just prints hello
11    cout << "hello" << endl;
12
13    auto end = chrono::high_resolution_clock::now();
14
15    double duration = chrono::duration<double>(end - start).count();
16    cout << "SECONDS: " << duration << endl;
17
18    return 0;
19 }
```

First, get the current time from your computer's most precise clock by using the **now()** function in the chrono library.

This statement asks the system for the current time, captures it as a very precise time point – down to microseconds or even nanoseconds (depending on your system), and returns it as a time_point object (like a timestamp) that is stored in the **start** variable.

```
hello
SECONDS: 0.000503
```


<chrono> library

```
1 #include <iostream>
2 #include <chrono>
3 using namespace std;
4
5 int main() {
6
7     auto start = chrono::high_resolution_clock::now();
8
9     //all the code you are timing
10    //this sample program just prints hello
11    cout << "hello" << endl;
12
13    auto end = chrono::high_resolution_clock::now();
14
15    double duration = chrono::duration<double>(end - start).count();
16    cout << "SECONDS: " << duration << endl;
17
18    return 0;
19 }
```

Second, get the current time using the **now()** function (again) after the code you are trying to measure elapsed time.

This statement asks the system for the current time, captures it as a very precise time point – down to microseconds or even nanoseconds (depending on your system), and returns it as a **time_point** object (like a timestamp) that is stored in the **end** variable.

```
hello
SECONDS: 0.000503
```

<chrono> library

```
1 #include <iostream>
2 #include <chrono>
3 using namespace std;
4
5 int main() {
6
7     auto start = chrono::high_resolution_clock::now();
8
9     //all the code you are timing
10    //this sample program just prints hello
11    cout << "hello" << endl;
12
13    auto end = chrono::high_resolution_clock::now();
14
15    double duration = chrono::duration<double>(end - start).count();
16    cout << "SECONDS: " << duration << endl;
17
18    return 0;
19 }
```

Last, calculate the time elapsed by subtracting the start time from the end and call the count() function to translate that to seconds.

```
hello
SECONDS: 0.000503
```

end – start gives you a duration (the amount of time between two time_points. Then, chrono::duration<double>() converts the raw duration into seconds, using double as the data type. Last, .count() extracts the number of seconds (as a double) from that duration.



10 Most Common OpenMP Directives

OpenMP Directive	Purpose
#pragma omp parallel	Starts a parallel region; creates multiple threads.
#pragma omp for	Divides a loop among threads inside a parallel region.
#pragma omp parallel for	Combines 'parallel' and 'for' to simplify syntax.
#pragma omp sections	Splits work into separate code blocks to run in parallel.
#pragma omp section	Marks each block of work inside 'sections'.
#pragma omp single	Ensures that only one thread runs a block of code.
#pragma omp master	Only the master thread runs the block.
#pragma omp critical	Ensures mutual exclusion for a code block (avoids race conditions).
#pragma omp barrier	All threads wait here until everyone reaches this point.
#pragma omp atomic	Performs simple operations on a shared variable atomically.

END

